

杰里 AC7076A3 SDK 与 ARM 工程对照

从项目配置、编译链接到运行入口与烧录包

基于当前 AC707N 3.2.1 SDK / Board1 工程实际文件核对

先建立这张脑图



核心认识

这个仓库不是单一的 **main.c + startup.s + HAL** 工程。可视化配置、条件编译、链接段注册、固件打包共同决定最终固件。只看到某个 C 文件存在，不能推断它一定参与本次构建或已经在板上运行。

本文范围：解释当前工程的组织与排查路径，供熟悉 STM32 / Cortex-M 裸机或 RTOS 工程的开发者快速定位。硬件例子采用本板 PB0 LED 与 360×360 JD9855 QSPI 屏；屏幕当前能点亮背光，但画面链路仍未完成验证。

阅读建议：先看目录图，再按“配置 → 构建 → 启动 → 外设”的顺序追踪一项功能。最后一页给出查找入口与核对清单。

以下是这份仓库中各目录的职责；不同目录不等于同一个硬件镜像。

从仓库根目录看

位置	本工程里的作用	熟悉 ARM 工程时可类比
project.jlproj src/	JLStudio 工程元数据、板级/功能/LCD 配置 JSON、音频 流程和资源输入。	配置工具工程与资源源文件
SDK/apps/watch/	手表应用主线：app_main、board、bringup、UI、模式 和业务。	App / Board / User
SDK/apps/common/	应用共享的 USB、设备、文件系统、更新等模块。	Middlewares / 通用组件
SDK/cpu/br35/	平台初始化、底层驱动、链接脚本源与下载规则。	CMSIS + BSP + linker script
SDK/cpu/br35/liba/	平台、媒体、设备等预编译 .a 库；并非每个实现都有可见 源码。	供应商静态库
SDK/interface/	SDK 公共接口头文件及链接段定义。	HAL API / 框架接口
SDK/audio/	音频栈及相关平台实现。	音频中间件
SDK/build/	构建规则与按宏生成的源文件清单。	构建系统/CMake 生成目录
SDK/tools/	工具程序和本板 build_ac7076a3_demo.ps1 包装脚本。	构建/烧录脚本
loader/ • SensorHub/	各自有 Makefile/工程文件的旁支工程；不能默认算进 watch app。	独立 bootloader / 协处理器工程
output/ • doc/	下载产物、构建记录；原理图和调试文档。	Artifacts / Docs

容易看错的名字

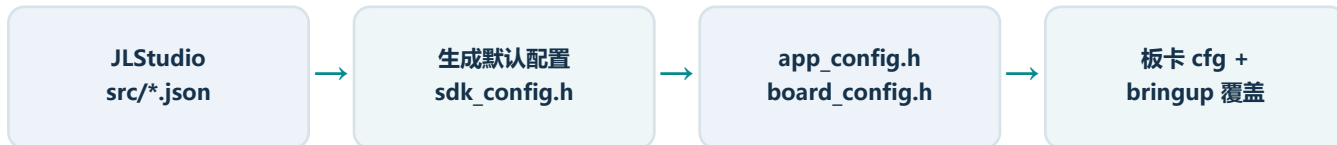
AC707N 是这份 JLStudio 工程采用的芯片系列标识，**br35** 是 SDK 平台目录；当前实物为 AC7076A3。

SDK/AC707N.cbp 是 IDE 视图，实际命令行编译入口是 **SDK/Makefile** 和本板包装脚本。

代码依据 project.jlproj:1-76；SDK/Makefile:15-26,206-216；SDK/build/README；SDK/tools/build_ac7076a3_demo.ps1:1-22；SDK/cpu/br35/liba/。

这套 SDK 的宏不是只在一个头文件里定义；应按包含顺序和预处理结果判断。

两条配置链在编译前汇合



实际包含顺序：app_config.h 引入 board_config.h；后者先读可视化工具生成的 sdk_config.h，再读本板 ac7076a3_demo_config.h、选中的板级 cfg，最后读 ac7076a3_demo_overrides.h。因此要看“最后生效值”，不能只搜到第一个 #define 就下结论。

本板的一个具体例子

层次	看到的值 / 行为	含义
JLStudio 生成默认	sdk_config.h: Flash 0x800000; UI=1。	原手表工程按 8 MiB 与完整 UI 配置。
Board1 Demo 覆盖	demo_config.h: Flash 0x400000; UI=0。	本板 4 MiB，先跑独立外设验证。
板级分区条件	无 UI 时禁用内置 SDFILE / VIRFAT，保留 FDB DATA 区。	资源布局随宏变化。
LCD 局部再使能	DEMO_LCD_ENABLE 控制屏幕驱动与任务； ui_app_def.h 也参与 LCD 宏重写。	“关手表 UI” 不等于 “禁用所有 LCD 代码”。

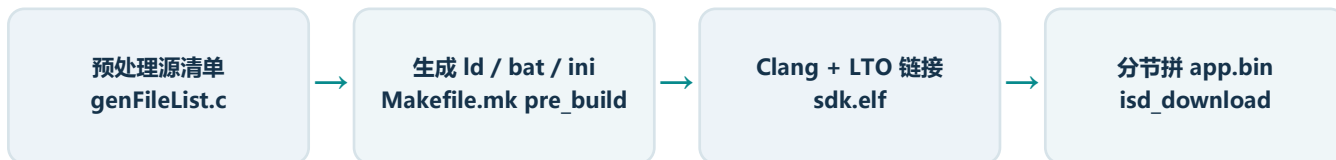
编辑原则

产品级图形配置改 **JLStudio 的源配置**；板级引脚/分区改对应 **board cfg** 或本板覆盖层；外设验证改 **bringup**。
sdk_config.h、**fileList.mk**、**sdk.ld**、**download.bat**、**isd_config.ini** 中的生成结果主要用于核对，直接修改通常会被下一次生成覆盖。

代码依据 SDK/apps/watch/include/app_config.h:8；board/br35/board_config.h:4-25；board/br35/sdk_config.h:36-39,207-218；bringup/ac7076a3_demo_config.h:9-22；board_ac707n_demo_cfg.h:95-111。

本工程的 ELF 还要经过分节提取、资源合并与下载器处理。

实际链路



环节	输入 → 输出	为什么与常见 ARM 工程不同
选择源文件	genFileList.c → build/fileList.mk, 再由 Makefile.mk 引入。	目录里有文件, 并不代表本配置会编译它。
生成构建参数	sdk_ld.c → sdk.ld; download.c → download.bat; isd_config_rule.c → isd_config.ini。	链接/烧录规则也经过 C 预处理。
编译链接	C:/JL/pi32 工具链; -target pi32v2 -mcpu=r3; 输出 cpu/br35/tools/sdk.elf。	不是 ARM GCC / Cortex-M 目标。
固件后处理	从 ELF 提取 .text/.data/overlay 等, 拼成 app.bin。	不能把 sdk.elf 直接复制到 U 盘当固件。
下载与封装	isd_download 使用 app.bin、uboot、cfg_tool.bin、stream.bin、KEY 等。	程序、资源、Flash 规则要配套。

本板推荐的两个命令

```

# 只编译 Screen 配置（不会运行下载批处理）
& .\SDK\tools\build_ac7076a3_demo.ps1 -Profile Screen

# 板子已进入 BR35 UBOOT 后，显式下载
& .\SDK\tools\build_ac7076a3_demo.ps1 -Profile Screen -Download -KeyFile 'C:\JL\keys\AC707N.key'

```

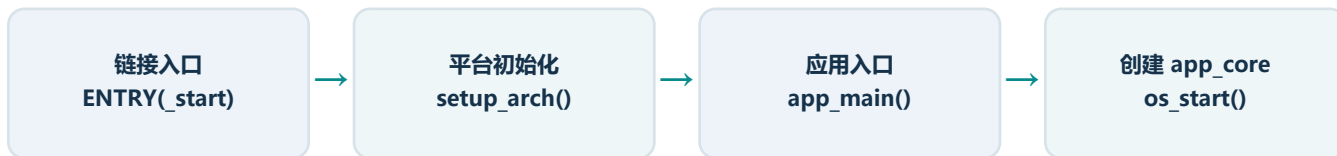
这里最容易误操作

SDK 原生 **make** 的默认 **all** 会走 post-build, 并可能直接调用下载脚本; 本板包装脚本改用 **build-only -B**, 只有加 **-Download** 才下载。-B 保证切换 -D 宏后对象文件重新编译。

代码依据 SDK/Makefile:15-26,69-72,206-216; SDK/build/Makefile.mk:909,1150-1168; SDK/cpu/br35/tools/download.bat:47-60; SDK/tools/build_ac7076a3_demo.ps1:17-42,66-128。

不要在仓库里只找 `main()`；运行时还包含启动代码、OS 任务和链接段注册。

可在当前源码中确认的入口链



链接脚本指定 `_start`；可见 C 源码中的 `setup_arch()` 初始化内存、看门狗、时钟与 OS 后调用 `app_main()`。`_start` 到 `setup_arch` 的中间细节没有在当前可见 C/汇编源码中核实，因此这里不推断 ROM 的具体跳转流程。

构建配置	app_core 执行	结果
原手表应用	<code>app_task_loop()</code> → <code>app_task_init()</code> → <code>initcall</code> / <code>board_init</code> → <code>APP_MODE</code> 循环。	完整手表业务与模式切换。
Board1 外设 Demo	<code>ac7076a3_demo_task()</code> , PB0 每约 1 s 切换电平；使能屏幕时再创建 <code>lcd_demo</code> 。	绕过原版 <code>app_task_init</code> ，用最少路径验证板子。

为何找不到一个显式调用某驱动的地方？

SDK 使用 **注册宏 + 特定链接段**：例如 `early_initcall` / `platform_initcall` / `module_initcall` 将函数指针放进对应段，初始化阶段遍历；`REGISTER_DEVICES`、`REGISTER_LCD_DEVICE` 把描述结构放进设备段。链接脚本用 **KEEP** 留住这些段，运行时按表寻找。

与常见 STM32 裸机工程的差异

常见路径是 `startup` → `SystemInit` → `main` → 手动 `HAL_Init` / `MX_xxx_Init`。本 SDK 的应用逻辑在 OS 任务中；某功能是否生效还要同时看宏、源文件是否编进来、注册段是否保留，以及运行时是否真的启动该任务。

代码依据 SDK/cpu/br35/sdk.ld:370；SDK/cpu/br35/setup.c:309-393；SDK/apps/watch/app_main.c:340-375,672-820；SDK/interface/system/init.h:12-51；SDK/interface/ui/ui.ld:96-102。

这个例子展示“宏 → 板级数据 → 驱动注册 → 通用 LCD 框架 → Demo 任务”的完整路径。

顺序	去哪里找	这一步实际决定什么
1 · 宏	bringup/ac7076a3_demo_config.h; bringup/ac7076a3_demo_overrides.h; ui/ui_app_def.h。	DEMO_LCD_ENABLE、LCD 宏、logo 匹配方式。
2 · 板级	board_ac707n_demo.c 中的 lcd_spi_data 与 ui_cfg_data。	Reset、TE、背光等平台引脚。
3 · 面板描述	bringup/lcd_qspi_jd9855.c 中的 jd9855_param 与 REGISTER_LCD_DEVICE。	360×360、RGB565、QSPI 模式/命令、初始化表、复位。
4 · 框架	cpu/br35/ui_driver/lcd_drive/lcd_drive.c。	依据 logo 选设备，配置 DBI，发送初始化命令，提供 LCD API。
5 · 验证任务	app_main.c 的 lcd_demo 任务表； bringup/ac7076a3_demo.c 与 lcd_qspi_jd9855.c。	启动/定时调用 LCD API 推测试色。

当前屏幕状态应这样解读

PB0 LED 闪烁说明 Demo 主任务还在跑；**背光亮**说明背光路径可工作；两者都不证明 JD9855 已接受初始化命令或像素数据。当前初始化表是暂用的模组参数，模组型号尚不明确，画面链路仍需验证。

为什么只知道 JD9855 和 360×360 还不够？ 同驱动 IC 的模组可能在复位/供电顺序、写入命令、像素格式、扫描方向或 QSPI 传输参数上不同。驱动注册成功是软件条件；显示纯色还需要面板实际响应。

一个关键硬件抽象：此 SDK 的屏幕推送走专用 **DBI/LCD** 模块，板级源码也注明它不是普通 SPI 控制器；排查时应先看 DBI 参数和固定引脚映射，再看普通 SPI 例程。

代码依据 SDK/apps/watch/bringup/ac7076a3_demo_config.h:54-88; bringup/lcd_qspi_jd9855.c:9-74,85-112;
board/br35/board_ac707n_demo/board_ac707n_demo.c:214-244; SDK/cpu/br35/ui_driver/lcd_drive/lcd_drive.c:616-696。

这里只对照常见 Cortex-M 裸机/RTOS 项目的阅读习惯，不把所有 ARM 项目概括成一种模式。

你熟悉的入口	本工程对应位置	应该如何理解
.ioc / .uvprojx	project.jlproj + src/*.json; 另有 AC707N.cbp。	图形配置和 IDE 视图，不是唯一编译依据。
startup.s / main.c	sdk.ld 的 _start; setup_arch(); apps/watch/app_main.c。	应用从 OS 任务而非单个 while(1) 展开。
HAL_Msplnit / pinmux	board_config.h、board_*_cfg.h、board_*_demo.c。	宏、板级平台数据、注册表共同生效。
源文件在 IDE 项目树	build/genFileList.c → fileList.mk → Makefile.mk。	按宏选择源文件；以生成清单为准。
静态驱动调用	initcall、REGISTER_DEVICES、 REGISTER_LCD_DEVICE。	链接段 + 遍历式发现。
.elf / .hex / .bin	sdk.elf → 分节 → app.bin → jl_isd.fw / update.ufw。	下载包还需 boot、配置资源、Flash 参数/KEY。
ARM GCC / armclang	C:/JL/pi32 的 Clang，-target pi32v2 -mcpu=r3。	不能套用 Cortex-M 的启动文件与链接参数。

一个实用判断法

看到陌生的宏或注册结构时，依次追问：**它在哪个头文件最终定义？本配置的 fileList.mk 是否包含实现文件？链接脚本是否保留对应段？运行时是哪个任务或 initcall 使用它？** 这比只看单个函数更快。

代码依据 project.jlproj:1-76；SDK/Makefile:15-26,69-72,206-216；SDK/build/Makefile.mk:909,1154-1168；SDK/interface/system/init.h:12-51；SDK/apps/watch/app_main.c:794-820。

按“真实生效值”和“实际构建产物”排查，可以避免在生成文件里兜圈子。

要做的事	优先查看	核对证据
改 PB0 LED / 按键 / I2C	bringup/ac7076a3_demo_config.h、ac7076a3_demo.c；板级 cfg。	宏值、引脚、电平、任务是否运行。
改 JD9855 屏	bringup/lcd_qspi_jd9855.c、jd9855_panel_init.h；板级 lcd_spi_data。	初始化表、DBI 模式、引脚及测试色调用。
改手表模式 / UI	apps/watch/app_main.c、mode/、ui/；project.jlproj + src/。	确认是否仍启用 Board1 Demo；资源是否重导出。
改 Flash / 资源	demo_config.h、board_*_cfg.h、isd_config_rule.c。	生成的 isd_config.ini 与 4 MiB 实机匹配。
查“代码没生效”	genFileList.c、fileList.mk、Makefile.mk、build 日志。	源文件已编译、宏已重新构建、ELF 时间/哈希更新。
查“下载失败”	tools/build_ac7076a3_demo.ps1、download.bat、output/bringup/download-*.log。	先看首个 ERROR、设备容量、Download completed。

本板的最短核对顺序

1 原理图核对供电和引脚；2 看最终宏及生成的 fileList.mk；3 用包装脚本 **-Profile Basic / Screen** 编译；4 查 build 日志和 sdk.elf；5 进入 BR35 UBOOT 后显式 **-Download**；6 用 PB0 LED、屏幕实际画面和下载日志分别证明运行、外设与烧录。

4 MiB 是本板的硬约束

原始可视化配置仍有 **8 MiB** 痕迹；包装脚本禁止把 **Original** 配置下载到当前 4 MiB 板。当前 4 MiB Demo 关闭完整手表 UI 资源布局；这不等于完整产品 UI 已能适配 4 MiB。

代码依据 SDK/tools/build_ac7076a3_demo.ps1:17-42,66-128；SDK/apps/watch/bringup/ac7076a3_demo_config.h:9-22；SDK/apps/watch/board/br35/board_ac707n_demo/board_ac707n_demo_cfg.h:95-111；output/bringup/download-Screen.log:121-133。